AI(API) 설정 안내 — 센터 담당자용

「제주 AI·빅데이터 활용 AI Agent 개발 챌린지」 제출작 **오름(OREUM)**

> 오름은 **OpenAI `gpt-6-luna` · 추론 강도 `high`** 로 전 과정을 검증했습니다. 센터는 **API 키 하나만** `.env` 에 넣으면 됩니다. 키가 없거나 연결이 안 되어도 분석·반출 검사·보고서는 모두 동작하고, AI가 쓰던 문장 자리에 규칙 기반 문장이 들어갑니다.

---

1. 한눈에 보는 설정

항목	값
제공자	OpenAI (`OREUM_LLM_PROVIDER=openai` , 기본값)
API	**Responses API** — `POST https://api.openai.com/v1/responses`
모델	**`gpt-6-luna`** (`OREUM_LLM_MODEL` , 기본값)
추론 강도	**`high`** — 질문 해석 `OREUM_LLM_EFFORT_PLAN` , 보고서·후속 질문 `OREUM_LLM_EFFORT_WRITE` (기본값)
응답 형식	모든 호출이 **엄격 JSON 스키마**(`text.format.type = json_schema` , `strict = true`)
SDK	`openai==3.46.0` (requirements.txt, install.bat이 자동 설치)
키	`.env` 의 **`OPENAI_API_KEY`** — 화면·보고서·로그에 표시되지 않음, 제출물에 미포함
허용 주소	`api.openai.com` (HTTPS 443)
가격(2026.11)	입력 100만 토큰당 0.10달러 · 출력 0.50달러 → 보고서 1건 약 5~10원, 후속 질문 1건 약 2~5원(추정)

2. 설정 순서(최초 1회, 약 5분)

1. **키 발급**: https://platform.openai.com → 로그인 → **Settings → Billing**에서 결제 수단 등록(선불 충전) → **Settings → API keys → Create new secret key**(이름 예: `jeju-bigdata-center-oreum`) → `sk-...` 복사(한 번만 표시)
2. 키를 만든 **프로젝트에서 `gpt-6-luna` 사용이 허용**되어 있어야 합니다(프로젝트 설정의 모델 권한이 '전체 허용'이면 그대로).
3. (권장) **Settings → Limits**에서 월 사용 한도(예: 1만 원) 설정
4. **키 입력**: 오름 폴더의 `.env` 를 메모장으로 열어 `OPENAI_API_KEY=` 뒤에 붙여 넣고 저장(앞뒤 공백 없이). 나머지 AI 항목(`OREUM_LLM_PROVIDER=openai` , `OREUM_LLM_MODEL=gpt-6-luna` , `OREUM_LLM_EFFORT_*=high`)은 **바꾸지 않습니다.**
5. **연결 점검**: **`check_ai.bat` 더블클릭** → `[정상]` 이 나오면 끝입니다. 이 점검은 오름이 실제로 쓰는 것과 **같은 형식**(Responses API · 추론 강도 · 엄격 JSON 스키마)으로 요청을 1번 보내 확인합니다(비용 1원 미만).
6. `run.bat` 으로 실행 → 분석 1건 후 왼쪽 메뉴의 〈AI 전송 기록〉에 호출 기록(모델·토큰·비용)이 보이면 실제 운영 연결까지 확인된 것입니다.

3. 오름이 보내는 요청(코드와 동일)

오름의 모든 AI 호출은 `app/agent/llm.py` 의 아래 한 가지 형태입니다.

```
from openai import OpenAI

client = OpenAI()                                    # OPENAI_API_KEY 환경변수 사용
r = client.responses.create(
    model="gpt-6-luna",
    instructions=SYSTEM,                             # 역할·규칙(예: 수치는 근거 표 값만 사용)
    input=USER,                                      # 질문 + 반출 검사를 통과한 집계표
    reasoning={"effort": "high"},
    max_output_tokens=16000,
    text={"format": {"type": "json_schema", "name": "oreum_output", "schema": SCHEMA, "strict": True}},
)
answer = json.loads(r.output_text)
```

같은 요청을 HTTP로 쓰면:

```
POST https://api.openai.com/v1/responses
Authorization: Bearer $OPENAI_API_KEY
```

```
Content-Type: application/json

{"model": "gpt-6-luna", "instructions": "...", "input": "...", "reasoning": {"effort": "high"},
 "max_output_tokens": 16000,
 "text": {"format": {"type": "json_schema", "name": "oreum_output", "schema": {...}, "strict": true}}}
```

단계	호출 이름	횟수	보내는 것	받는 것
계획 Agent (Planner)	`plan_1`, `plan_2`, `plan_3`	2~3회 (단계마다 1회, 최대 3회)	이용자 질문, 오늘 날짜, 도구 목록 (이름·설명·입력 형식), 앞 단계 도구 결과 (지역·업종 표준 이름, 축제 회차·날짜, 데이터 명세 요약: ID·이름·기간·갱신 주기·반출 등급·설명 첫 문장)	도구 호출(`action: call_tool`, 도구명·인자) 또는 최종 계획(`action: finalize`, 분석 모드·지역·업종·축제·선택 데이터셋 ID·사유·설명 등)
해석 (Narrator)	`narrative` (검증 실패 시 `narrative_fix`)	1회 (수치 검증 실패 시 1회 추가)	질문·분석 조건 + **반출 검사 통과 집계표**	보고서 문장 (`headline`·`summary`·`findings`·`recommendations`·`business_text`·`limitations`·`export_purpose`)
후속 질문 (Follow-up Agent)	`followup_1` …	질문당 1~6회	질문 + 이전 근거 + 도구 결과 (검사 통과본)	도구 호출(`action: call_tool`) 또는 최종 답변(`action: answer`)

- 모든 호출은 **같은 모델(`OREUM_LLM_MODEL`, 기본 `gpt-6-luna`) · 같은 추론 강도(`high`) · 엄격 JSON 스키마**로 수행되며, 호출 이름(`purpose`)이 화면의 「AI 전송 기록」에 그대로 남습니다.
- **프롬프트 원문 위치**: `app/agent/prompts.py`
- **도구 5종 (`app/agent/plan_tools.py`)**: `find_region`, `find_industry`, `find_festival`, `search_datasets`, `get_dataset` — 이 PC의 사전·명세서 카탈로그(센터 71종 + 제주데이터허브 9종)만 조회.
- **계획 Agent가 보내는 것과 보내지 않는 것**: 모두 공개 명세·사전 정보이며, **데이터 값(원본 행·집계값)은 일절 보내지 않습니다.**
- **보내지 않는 것**: 원본 행(센터 CSV·허브 원자료), 10 미만 원값, 반출 검사에서 차단된 표, API 키, 파일 경로.
- 실제 전송 원문 예시: `docs/AI_전송_예시.md` · 호출마다 화면 〈AI 전송 기록〉에 보낸 내용·받은 내용·토큰·비용이 남습니다.
- AI가 고른 도구는 **센터 PC에서 계산**되고, 결과도 반출 검사를 거친 뒤에만 다음 요청에 붙습니다.

4. 어떻게 검증했나(개발 중)

- 개발 환경에는 API 키가 없어, **같은 모델(`gpt-6-luna`)·같은 추론 강도(`high`)·같은 프롬프트·같은 JSON 스키마**를 OpenAI Codex CLI(개발자 개인 ChatGPT 로그인, `OREUM_LLM_PROVIDER=codex` — **개발 검증 전용**, 센터 운영에는 쓰지 않음)로 실행했습니다.
- 대표 3개 시나리오 + 다양한 질문 + 후속 질문 도구 전부를 실제 화면에서 끝까지 실행: AI 호출 오류 0건, 문장 속 수치 근거 대조 통과 (상세: `docs/결과_설명자료.md` 「AI 모드 실행 검증」).
- OpenAI API 쪽은 실제 서버에 연결되어 **인증 단계까지** 확인했습니다(잘못된 키 → 401 → 규칙 기반으로 자동 전환). **키를 넣은 뒤 `check_ai.bat` 으로 요청 형식이 그대로 받아들여지는지 꼭 한 번 확인**해 주세요.
- AI 문장은 호출마다 표현이 조금씩 다를 수 있습니다. **수치·표·반출 판정은 코드가 계산하므로 항상 같습니다.**
- 계획 Agent(도구 사용) 정확도와 대표 시나리오 실행 시간·비용은 docs/eval/ 의 측정 결과를 참고.

5. 문제 해결

`check_ai.bat` 또는 〈AI 전송 기록〉의 오류 문구로 원인을 찾습니다. 어떤 경우에도 분석·보고서는 규칙 기반으로 계속 만들어집니다.

오류 문구	원인	조치
API 키 인증 실패	키가 틀림·만료·공백 포함	`.env` 의 `OPENAI_API_KEY` 다시 붙여 넣기
요청 한도 초과	잔액 부족·사용 한도 도달·분당 한도	Billing 잔액, Limits 확인 후 잠시 뒤 재시도
네트워크 연결 실패	방화벽·DLP 차단	`api.openai.com` (HTTPS 443) 허용 목록 등록
API 오류(404)	모델 이름 오타·모델 권한 없음	`OREUM_LLM_MODEL=gpt-6-luna` 확인, 프로젝트 모델 권한 확인
API 오류(400): …	모델이 요청 형식(추론 강도·스키마)을 받지 않음	기본값(`gpt-6-luna` · `high`)으로 되돌리기. 다른 모델은 Responses API· `reasoning.effort` ·엄격 JSON 스키마를 모두 지원해야 함
API 오류(403)	프로젝트·조직 권한, 지역 제한	키를 만든 프로젝트 권한 확인
응답이 길이 제한에 걸림	출력 토큰 한도	드묾 — 질문을 나눠서 다시 실행

- AI를 잠시 끄려면 `.env` 에서 `OREUM_LLM_ENABLED=0` .

6. 다른 모델·제공자

바꾸려는 것	설정
더 높은 품질(비용 약 10배)	`OREUM_LLM_MODEL=gpt-6.1-terra`
응답을 더 빠르게	`OREUM_LLM_EFFORT_PLAN=medium` (질문 해석), 작성은 `high` 유지 권장
Claude 사용	`OREUM_LLM_PROVIDER=anthropic` , `OREUM_LLM_MODEL=claude-opus-5-5` , `ANTHROPIC_API_KEY=...` (https://console.anthropic.com), 허용 주소 `api.anthropic.com`

다른 모델로 바꾼 경우에도 `check_ai.bat` 으로 먼저 확인하세요. 오름이 검증한 설정은 **OpenAI `gpt-6-luna` · `high`** 입니다.